Introduction to Streaming – Structured & Deep Notes

What Have We Covered So Far

Before streaming, we learned:

- Transformations vs Actions
- Lazy evaluation
- Lineage & execution plans
- Catalyst Optimizer
- Joins (inner, outer, semi, anti, broadcast)
- Caching & partitioning
- Data skewness
- Writing to sinks

All of that was in a **batch-processing world**.

Now we move to something fundamentally different:

> 🔁 **Continuous data processing – Streaming**

Transition to Streaming

From Batch to Real-Time

Batch processing assumes:

- Data is **finite**
- Data is fully available before processing
- Job starts → runs → finishes

Streaming assumes:

- Data is **unbounded**
- Data arrives continuously
- Job never stops
- Results are continuously updated

🔎 Analogy by Domain

🏦 Banking

Batch: Detect fraud at end of day
Streaming: Detect fraud at transaction time

🛒 E-commerce

Batch: Yesterday's sales report
Streaming: Live sales dashboard

🖥️ IT Monitoring

Batch: Analyze logs after outage
Streaming: Real-time error alerts

🌡️ IoT

Batch: Analyze sensor data hourly
Streaming: Immediate anomaly detection

Why Batch Processing Falls Short

Batch introduces:

- Latency
- Delayed decisions
- Reprocessing of old data
- No real-time visibility

🔎 Real-Life Limitations

Fraud Detection

Fraud at 10:05 AM
 Batch runs at 12:00 PM
 Too late.

IT Monitoring

Server fails at 1:00 PM
 Logs analyzed at 5:00 PM
 Users suffer downtime.

Ride Sharing

Traffic analyzed hours later
 Drivers get no real-time help.

What Is Streaming?

Streaming is:

> Continuous processing of continuously arriving data.

Key idea:

- Data is never complete.
- Results are never final.
- Everything is incremental.

Streaming applications:

- Stay alive
- Listen continuously
- Process incrementally
- Update results dynamically

Batch vs Streaming : Core Comparison

Aspect	Batch	Streaming
Data	Finite	Infinite
Execution	Start → Stop	Continuous
Latency	High	Low
Result	Final	Evolving
Trigger	Manual / Scheduled	Data-driven
Mental Model	“Process everything once”	“Keep processing forever”

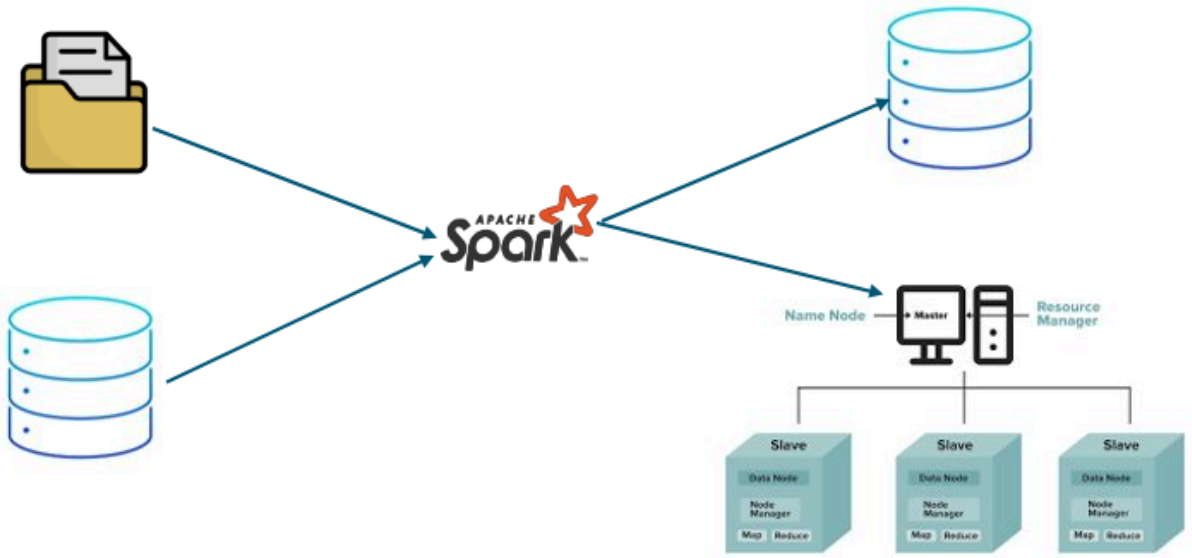
Batch Processing Architecture (Hadoop Ecosystem)

Phases

1️⃣ Source Systems
2️⃣ Data Ingestion (Sqoop, Flume)
3️⃣ HDFS Storage
4️⃣ Processing (Spark / MapReduce)
5️⃣ Output Storage
6️⃣ Analytics (Hive / BI)

Batch flow:

Source → HDFS → Spark → HDFS → Reports



Streaming Without Kafka (Direct Ingestion)

Simple streaming flow:

Source → Spark Streaming → Storage → Dashboard

Works for:

- Learning
- Small scale
- Low traffic systems

But has serious limitations.

The Problem with Direct Pulling (Very Important)

Scenario: Food Delivery App

Spark pulls data directly from database.

Everything works…

Until Spark crashes.

What Happens When Spark Fails?

- Orders keep coming
- Database keeps updating
- Spark is down
- No processing happens

When Spark restarts:

- It doesn’t know what it missed
- Data may be:

    - Skipped
    - Duplicated
    - Partially processed

Why This Happens

Source systems:

- Don’t track what Spark read
- Don’t replay safely
- May overwrite old logs

Spark:

- Has no safety buffer
- Has no memory of missed events

Business Impact

- Lost orders
- Incorrect revenue
- Broken dashboards
- SLA violations

Streaming becomes unreliable.

The Solution : Intermediate Layer

We introduce:

Message Queue / Messaging Layer

Instead of:

Source → Spark

We use:

Source → Message Queue → Spark

What Is a Message Queue?

A message queue:

- Receives events
- Stores them temporarily
- Allows safe consumption
- Ensures durability
- Enables replay

It decouples:

- Data production
- Data processing

Key Characteristics

- Durable storage
- Fault tolerant
- Replay capability

- Scalable
- Supports multiple consumers

🔎 Analogy (Food App Revisited)

Before:
Spark pulls directly → crash → lost data

After:
Order → Message Queue → Spark

If Spark crashes:
Events remain in queue
Spark resumes safely

No data loss.

Streaming Architecture with Kafka

Modern production flow:

Source → Kafka → Spark Structured Streaming → Storage → Dashboard

Phases

1️⃣ Source Systems
2️⃣ Kafka Ingestion
3️⃣ Stream Processing
4️⃣ Output Storage
5️⃣ Analytics

Streaming Processing Frameworks

1️⃣ Spark Streaming / Structured Streaming

- Micro-batch model
- Unified batch + streaming
- Easy for Spark users

2️⃣ Apache Flume

- Log ingestion tool
- Not full analytics engine

3️⃣ Apache Storm

- True event-by-event processing
- Very low latency

4️⃣ Apache Flink

- Streaming-first design
- Advanced event-time handling

High-Level Comparison

Framework	Strength
Spark	Analytics + streaming
Flink	Advanced streaming
Storm	Ultra low latency
Flume	Log ingestion

Messaging Frameworks

Messaging frameworks move data safely.

They do NOT analyze data.

🟢 Apache Kafka

- High throughput
- Durable
- Replay support
- Multiple consumers
- Production-grade

🔵 Apache Pulsar

- Cloud-native
- Multi-tenant

🟡 RabbitMQ

- Workflow-based messaging
- Task routing

🟠 Amazon Kinesis

- Managed cloud streaming

Streaming Architecture Components

Sources

Examples:

- ATM machines
- Mobile apps
- IoT sensors
- Logs

Characteristics:

- Continuous
- Bursty
- Unpredictable

Processing Engine

Examples:

- Spark
- Flink

Responsibilities:

- Apply logic
- Maintain state
- Handle failures

- Scale horizontally

Sinks

Examples:

- Dashboards
- Databases
- Alerts
- Data lakes

Purpose:
Turn events into business value.

Triggers

Triggers define:

- When micro-batches execute
- How often results update

They balance:

- Latency
- Resource usage

Checkpointing (Critical Concept)

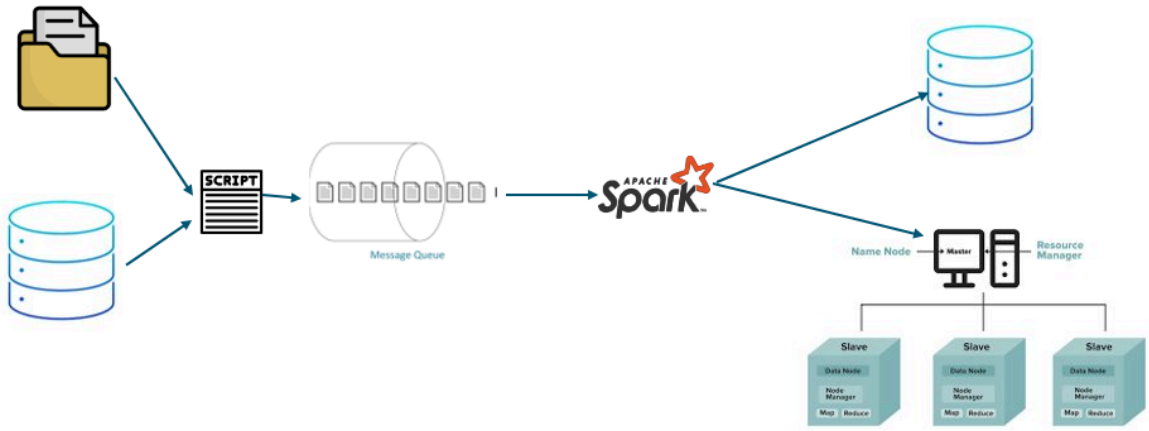
Checkpointing:

- Saves processing progress
- Stores internal state
- Enables restart recovery

🔎 Analogy

Like auto-save in Google Docs:

- Crash doesn't erase work
- You resume from last saved state



Core Structured Streaming Concepts

Structured Streaming treats streaming as:

> A continuously updating table

You write queries like batch.
Spark runs them continuously.

Streaming DataFrames

They look like batch DataFrames…

But:

- They keep growing
- Data keeps appending
- Queries keep updating

🔎 Real-Life Example

Attendance system:
Students enter → table grows
Query: total present
Result updates live.

Unbounded Data (Infinite Tables)

Batch:
Finite dataset

Streaming:
Infinite table

New rows are appended forever.

Results evolve continuously.

Micro-Batch Execution Model

Structured Streaming:

- Groups data into small batches
- Processes each batch using Spark engine
- Repeats continuously

Combines:

- Batch reliability
- Near real-time speed

🔎 Analogy

Customers arrive continuously
Cashier processes small groups

Efficient and stable.

Event Time vs Processing Time

Event Time

When event actually happened

Processing Time

When Spark processed it

They may differ due to:

- Network delays
- Retries

- Failures

Using wrong time causes:

- Wrong aggregations
- Incorrect windows

Exactly-Once Processing (Conceptual)

Goal:

Each event affects result exactly once.

No:

- Duplicates
- Missing records

Achieved using:

- Offset tracking
- Checkpointing
- Idempotent writes

Critical for:

- Banking
- Billing
- Finance

Streaming Sources

File-Based Streaming

Spark monitors directory for new files.

Good for:

- Learning
- Transitional systems

Not truly real-time.

Socket Source

Used for:

- Demo
- Classroom
- Learning

Limitations:

- No durability
- No replay
- Not production-ready

Kafka Source (Production Standard)

Kafka:

- Stores events durably
- Tracks offsets
- Supports replay
- Enables multiple consumers

When Spark restarts:

- Reads from last offset
- No data loss

Why Kafka Is Production Grade

- High throughput
- Fault tolerant
- Horizontally scalable
- Supports spikes

Source Characteristics

Rate of Data Arrival

Streaming systems must handle:

- Slow trickle
- Sudden spikes

- Unpredictable bursts

Example:
Flash sale → 10x traffic

System must absorb surge safely.

Schema Inference vs Predefined Schema

Schema Inference

- Auto-detect structure
- Good for experiments
- Risky in production

Predefined Schema

- Explicit structure
- Stable
- Predictable
- Safer

Production systems prefer predefined schemas.

🎯 Key Mental Shifts in Streaming

1. Data is never complete
2. Results are never final

3. Jobs never terminate
4. Time matters (event vs processing)
5. Fault tolerance is mandatory
6. Messaging layer is essential
7. Exactly-once guarantees are critical

🎯 Interview Questions from This Module

- Why does batch processing fail for real-time systems?
- Why do we need Kafka between source and Spark?
- What is unbounded data?
- What is micro-batch processing?
- Difference between event time and processing time?
- What is checkpointing?
- Why is exactly-once processing important?
- Why is Kafka production-grade?
- What happens if Spark crashes without checkpointing?
- What are triggers in Structured Streaming?

🔁 Final Big Picture

Streaming Architecture:

Sources
↓
Messaging Layer (Kafka)
↓
Processing Engine (Spark Structured Streaming)
↓
Sinks (DB, Dashboard, Alerts)
↓
Continuous Insights